FULL STACK DEVELOPMENT WITH MERN

ShopEZ

E-Commerce Web Application

College Name: Aditya College Of Engineering and Technology

Department: Department of Computer Science & Engineering

Course: Bachelor of Technology

Project Documentation

Submitted By

Name	Role
Md Hamim	Team Leader
Lokesh Tellamekala	Full Stack Developer
Pathivada Siva Vaishnavi	Frontend Developer
Upparapalli Bala Naga Satya Harini	Database Administrator
Neelima Siriki	API Development & Testing

Table of Contents

1. Introduction 03

2. Project Overview 04

3. System Architecture 05

4. Technology Stack 06

5. Installation & Setup 07

6. Folder Structure 08

7. Running the Application 09

8. API Documentation 10

9. Authentication 11

10. User Interface 12

11. Database Design 13

12. Testing 14

13. Known Issues 15

14. Future Enhancements 16

15. Conclusion 17

Project Overview

Purpose:

ShopEZ is a modern e-commerce web application developed using the MERN Stack (MongoDB, Express.js, React.js, and Node.js). It provides a simple, secure, and user-friendly online shopping platform for customers. Users can register, log in, browse products, search for items, view product details, add products to their shopping cart, and place orders securely. The application also provides an admin panel for managing products and orders efficiently. The primary goal of ShopEZ is to deliver a fast, responsive, and reliable online shopping experience.

Features:

- User Registration and Login
- Secure Authentication
- Product Browsing
- Product Search
- Product Details
- Shopping Cart
- Secure Checkout
- Order Confirmation
- Admin Dashboard
- Product Management
- Order Management
- Customer Reviews
- Responsive User Interface
- MongoDB Database Integration

Architecture

Frontend :

The frontend of ShopEZ is developed using React.js to create a responsive and interactive user interface. It allows users to register, log in, browse products, search for items, add products to the shopping cart, and place orders. React Router is used for navigation between different pages, while Axios is used to communicate with the backend APIs. HTML, CSS, and JavaScript are used to build a clean and user-friendly interface.

Backend:

The backend of **ShopEZ** is developed using **Node.js** and **Express.js**. It handles business logic, user authentication, product management, shopping cart operations, and order processing. The backend exposes RESTful APIs that receive requests from the React frontend, process the data, and communicate with the MongoDB database. JWT authentication is used to secure user access

Database:

MongoDB is used as the database for ShopEZ. It stores user accounts, product information, shopping cart details, and order records. The backend uses Mongoose to define schemas and interact with the database. MongoDB provides efficient storage, retrieval, updating, and deletion of data through CRUD operations.

Database Collections:

- Users
- Products
- Categories
- Cart
- Orders

Architecture Flow:

User

▼

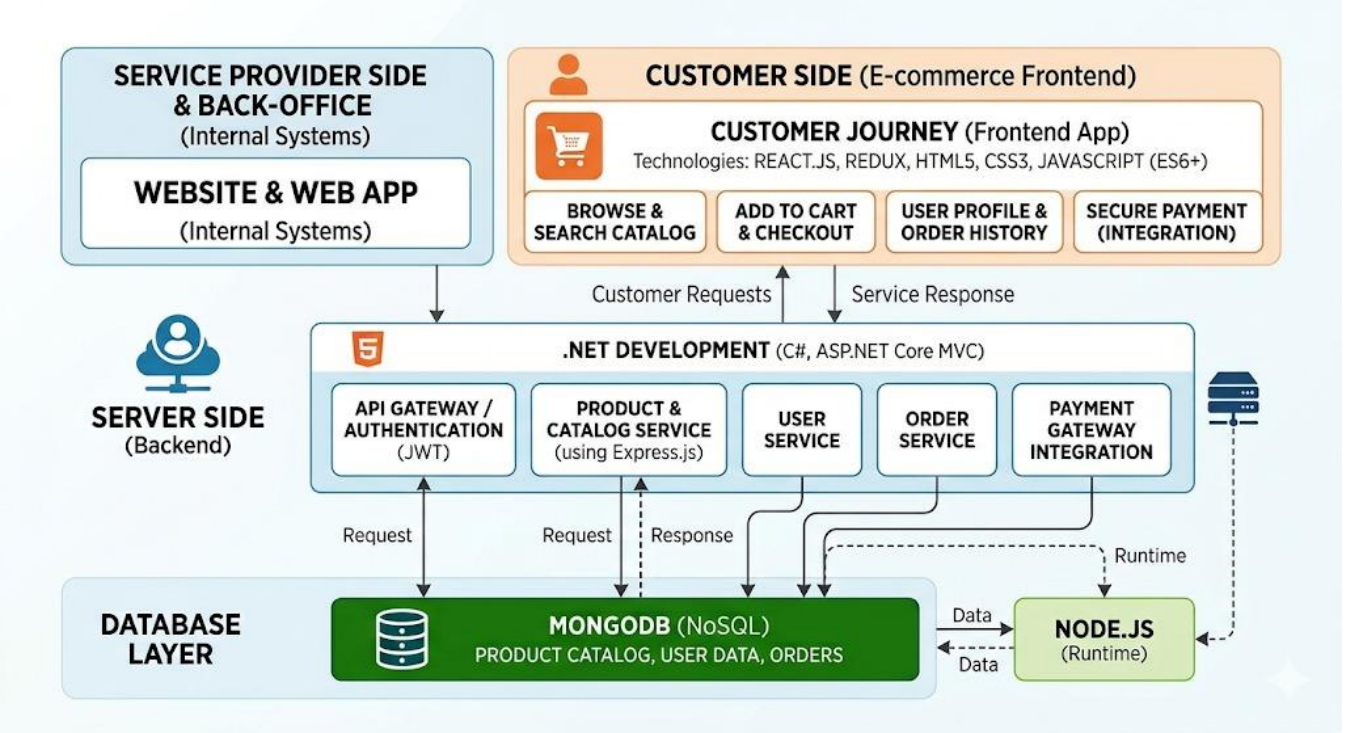
React.js Frontend
(Client)

▼

Node.js + Express.js
(Server)

▼

MongoDB Database



4.Setup Instructions

4.1 Prerequisites

- Node.js
- npm
- MongoDB Community Server
- Visual Studio Code
- Git
- Google Chrome or any modern web browser

4.2 Installation Steps

Step 1: Clone the Repository
Git clone : https://github.com/sivavaishnavi/shopze.git

Step 2: Open the Project Folder in Visual Studio Code
cd Shoze

Step 3: Install Frontend Dependencies
cd client
npm install
npm run dev

Step 4: Install Backend Dependencies
cd server
npm install
npm run dev

step 5: Environment Variables Setup

Create a .env file inside the server folder

MONGO_URI=mongodb+srv://vaishnavipathivada32_db_user:Aditya%40123@cluster0.5uh8juj
.mongodb.net/ShopZon e?appName=Cluster0

JWT_SECRET=1ec9ec4480b56b47a931df01cb81a4ab9f01a
61b1aaaf5beeb3dfcdd402fb4e2

PORT=8000

5. Folder Structure

5.1 Client Folder (Frontend)

Client

```
│
├── node_modules
├── index.html
├── package.json
├── package-lock.json
├── README.md
└── vite.config.js
```

Description:

- node_modules – Contains all installed frontend dependencies.
- index.html – Entry HTML file for the React application.
- package.json – Contains project information, dependencies, and scripts.
- package-lock.json – Stores the exact versions of installed packages.
- README.md – Contains project documentation and setup instructions.
- vite.config.js – Configuration file for the Vite development server.

5.2 Server Folder (Backend)

Server

├── config

├── Models

├── schemas.js

├── node_modules

├── .env

├── package.json

├── package-lock.json

└── server.js

Description

- **config** – Contains database configuration files.
- **Models** – Contains MongoDB models and schemas.
- **schemas.js** – Defines the database schema using Mongoose.
- **node_modules** – Contains installed backend dependencies.
- **.env** – Stores environment variables such as MongoDB URI and JWT secret.
- **package.json** – Contains backend dependencies and scripts.
- **package-lock.json** – Stores the exact versions of installed packages.
- **server.js** – Entry point of the backend server.

6. Running the Application

6.1 Start the Backend Server

Open a terminal and run:

cd server

npm start

Application Backend url: **http://localhost:8000**

6.2 Start the Frontend Server

Open another terminal, navigate to the client folder, and run:

cd client

npm run dev

Access the Application url: **http://localhost:5173**

7. API Documentation

7.1 Authentication APIS

Models	EndPoint	Description
Post	/api/users/register	Register a new User
Post	/api/users/login	User Login and JWT generation
Post	/api/users/profile	Get Logged-in user profile

Login Request:

POST : /api/users/login

Content-Type: application/json

```
{
  "email": "Neelu@123.com",
  "password": "Admin123"
}
```

Response

```
{
  "success": true,
  "token": "JWT_TOKEN",
  "user": {
    "_id": "12345",
    "name": "Admin",
    "email": "Neelu@123.com",
    "role": "admin"
  }
}
```

7.2 Product APIs

Method	EndPoint	Description
Get	/api/products	Fetch all products
Get	/api/products/:id	Get product details

Post	/api/products/add-product	Add a new product (Admin only)
Put	/api/products/:id	Add a new product (Admin only)

7.3 Cart APIs

Method	EndPoint	Description
Get	/api/cart	Get the user's cart
Post	/api/cart/add	Add a product to the cart
Put	/api/cart/update	Update product quantity
Delete	/api/cart/remove/:id	Update product quantity

7.4 Order APIS

Method	EndPoint	Description
Post	/api/orders/create	Create a new order
Get	/api/orders	Get the user’s order
Get	/api/orders/all	Get all orders

7.5 Admin APIS

Method	EndPoint	Description
Get	/api/admin/stats	View dashboard statistics

Get	**/api/admin/users**	**View dashboard statistics**
Get	**/api/admin/orders**	**View dashboard statistics**

8. Authentication:

ShopEZ uses **JSON Web Token (JWT)**-based authentication to secure user sessions and protect sensitive resources. User passwords are encrypted using **b crypt** before they are stored in the MongoDB database.

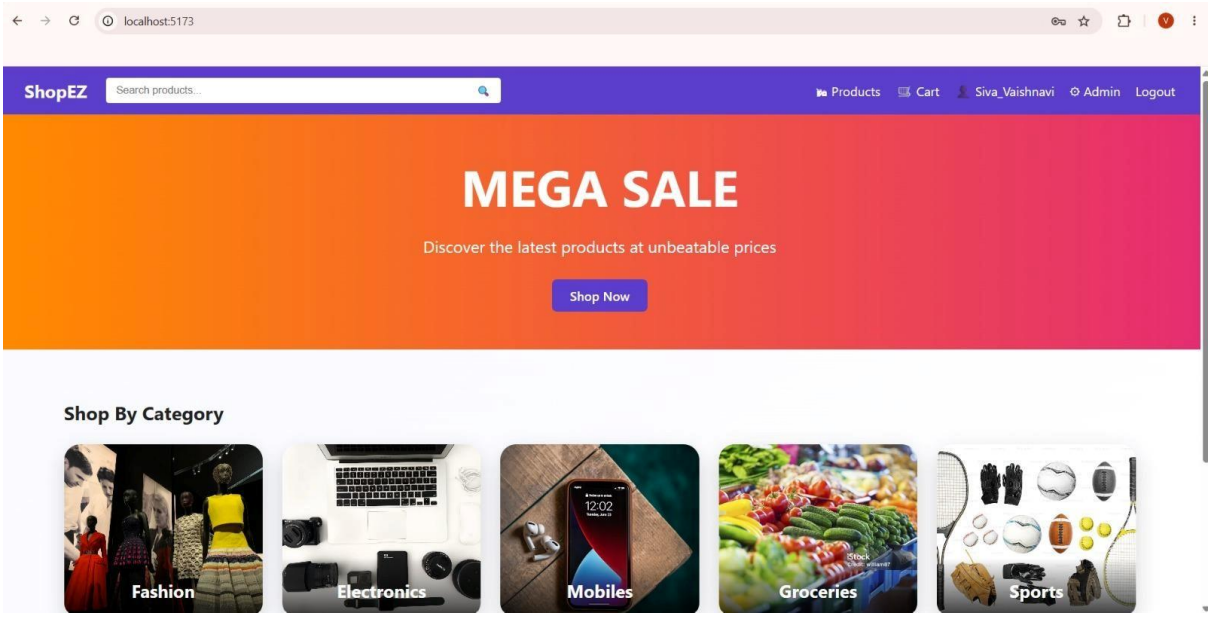
Authentication Flow

1. The user registers or logs in using an email address and password.
2. The backend validates the user's credentials.
3. Upon successful authentication, the server generates a JWT token.
4. The frontend stores the token in Local Storage.
5. The token is included in the Authorization header for every protected API request.
6. Backend middleware verifies the JWT before granting access to protected resources.
7. Admin-specific APIs are accessible only to authenticated users with the Admin role.

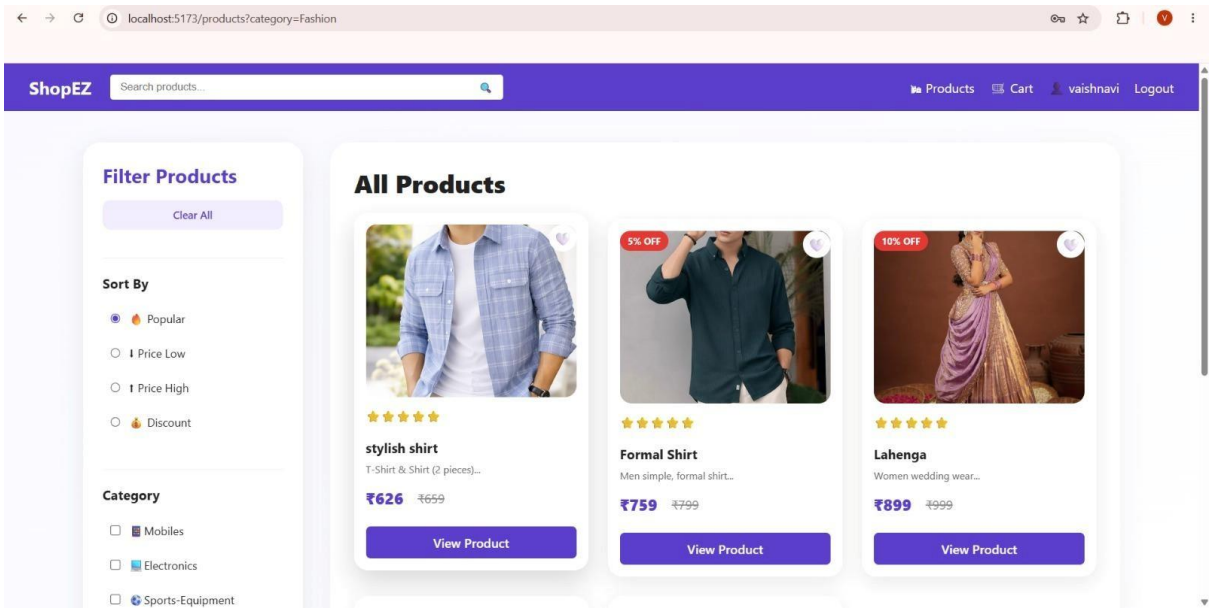
9. User Interface

The ShopEZ application provides a modern, responsive, and user-friendly interface developed using React.js. The application is designed to ensure a smooth shopping experience across desktops, tablets, and mobile devices

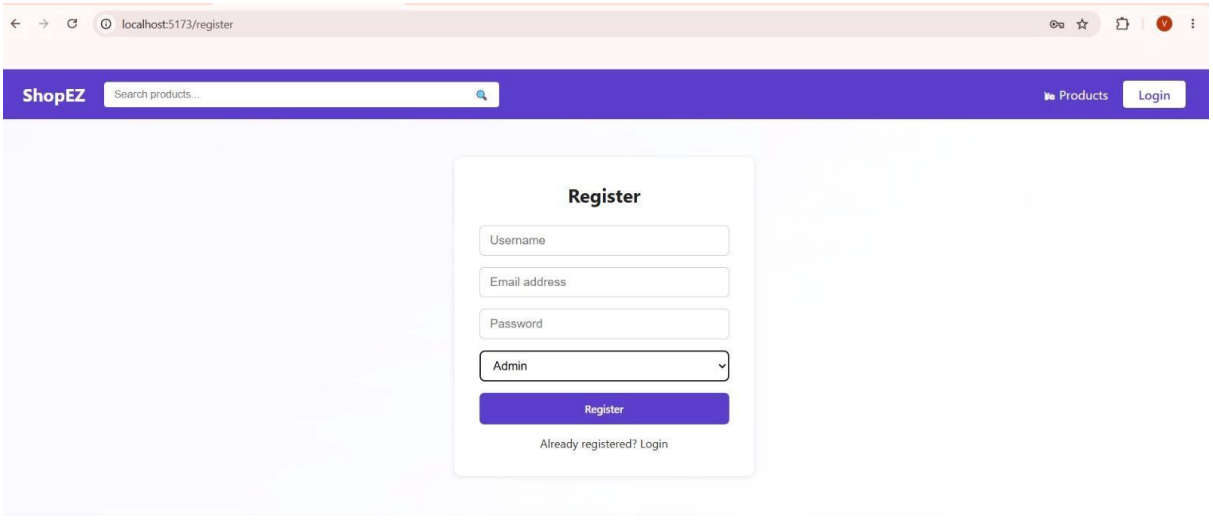
9.1: Home Page



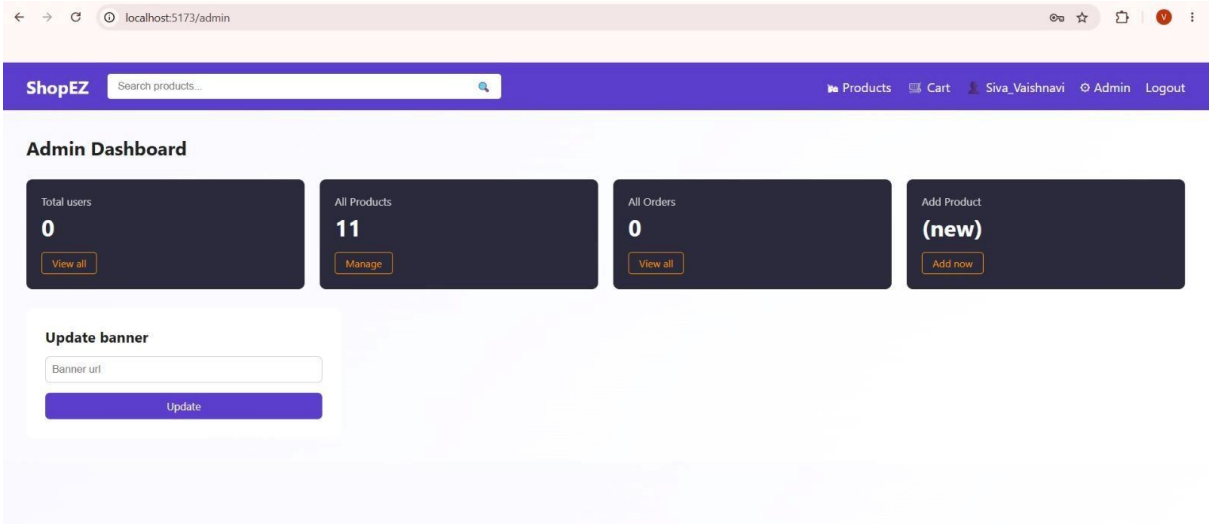
9.2: Product Listing Page



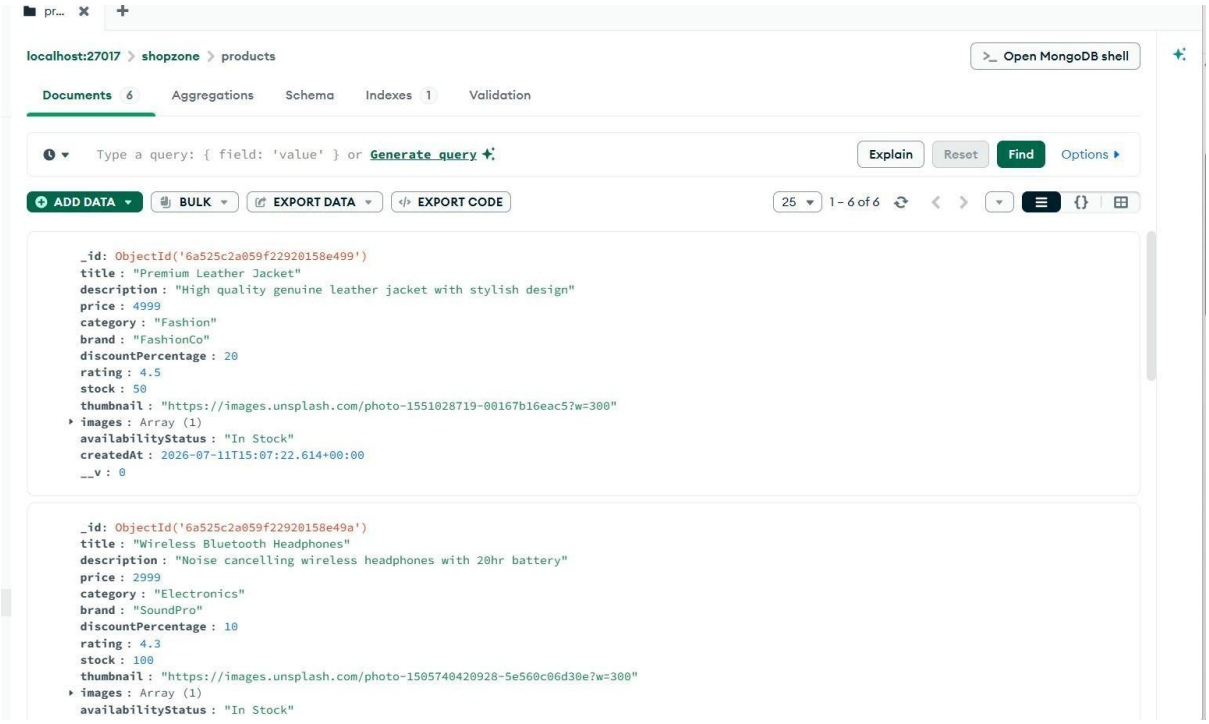
9.3: User Registration Page



9.4: Admin Dashboard



9.5: MongoDB Database



10. Testing

The ShopEZ application was tested to ensure that all major functionalities work correctly and provide a smooth user experience. Both frontend and backend modules were tested individually and together.

Module	**Test Case**	**Result**
User Registration	Register a new user	✅ Passed
User Login	Login with valid credentials	✅ Passed
Product Display	View all products	✅ Passed
Product Search	Search products	✅ Passed
Category Filter	Filter by category	✅ Passed

Shopping cart	Add items to cart	✅ Passed
Order Management	Place an order	✅ Passed
Admin Dashboard	Manage products	✅ Passed
MongoDB Database	Store & retrieve data	✅ Passed

12. Known Issues

The current version of ShopEZ has a few limitations that can be improved in future releases.

- Payment gateway integration is not implemented.
- Email notifications are not available.
- Product reviews and ratings cannot be submitted by users.
- Wishlist functionality is limited.
- Performance may decrease with a very large number of products.

13. Future Enhancements

The following enhancements can improve the ShopEZ application in future versions:

- Integrate Razorpay or Stripe payment gateway.
- Add order tracking functionality.
- Implement product reviews and ratings.
- Add AI-based product recommendations.
- Provide email and SMS notifications.
- Introduce coupon codes and discount offers.
- Implement Two-Factor Authentication (2FA).
- Develop Android and iOS mobile applications.
- Improve performance and scalability using cloud deployment.
- Add multilingual support.